

Section 1: Stack Data Structure (Questions 1-17)

Q1 [GATE CSE 2004]

An input sequence of elements 1, 2, 3, 4, 5 arrives in strict sequential order. Which of the following permutations cannot be obtained as an output sequence using a single stack?

- A) 1, 2, 3, 4, 5
- B) 2, 4, 3, 5, 1
- C) 3, 1, 2, 5, 4
- D) 4, 5, 3, 2, 1

Correct Answer: C) 3, 1, 2, 5, 4

Explanation: To get 3 as the first output, 1, 2, and 3 must be pushed in sequence. The stack contents are then [1, 2, 3] with 3 at the top. Once 3 is popped, the top element becomes 2. Therefore, it is structurally impossible to pop 1 before popping 2. Thus, the sequence 3, 1 cannot be achieved.

Q2 [GATE CSE 2013]

What is the worst-case time complexity of executing a sequence of n push and pop operations on an initially empty stack implemented using a dynamically resizing array that doubles its capacity whenever it becomes full?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n^2)$

Correct Answer: C) $O(n)$

Explanation: While an individual push triggering array reallocation takes linear time to copy existing elements, doubling happens infrequently at powers of 2. Summing up all copy steps yields a geometric series: $1+2+4+\dots+n \leq 2n$ operations. Through amortized analysis, the average cost per operation is $O(1)$, making the total worst-case time complexity for n operations $O(n)$.

Q3 [GATE CSE 2015]

Assume operators $+$, $-$, $*$ are left-associative, while exponentiation is right-associative. What is the correct postfix equivalent of the infix expression: $a + b * c - d ^ e ^ f$?

- A) $a\ bc^*+def^{^^}-$
- B) $abc+def^{^^}-$
- C) $ab+cdef^{^^}-$
- D) $abc*de+^{^f}-$

Correct Answer: A) $a\ bc^+def^{^^}-$*

Explanation: Due to right-associativity, $d \wedge e \wedge f$ is parsed from right to left as $(d \wedge (e \wedge f))$, converting to postfix as $def^{^^}$. Multiplication has higher precedence than addition, so $b * c$ becomes bc^* . The expression reduces to $a + bc^* - def^{^^}$. Left-to-right evaluation of equal-precedence $+$ and $-$ operations yields abc^*+ followed by the subtraction, resulting in $abc^*+def^{^^}-$.

Q4 [GATE CSE 2007]

What is the minimum number of stacks required to efficiently simulate a First-In-First-Out (FIFO) Queue data structure?

- A) 1 Stack
- B) 2 Stacks
- C) 3 Stacks
- D) 4 Stacks

Correct Answer: B) 2 Stacks

Explanation: A single stack reverses the arrival sequence due to its LIFO nature. By popping items out of the first stack and pushing them into a second stack, the original FIFO order is fully restored.

Q5 [GATE CSE 1998]

The postfix expression $6\ 2\ 3\ +\ *\ 3\ 8\ 2\ /\ /\ +$ is evaluated using a stack. What is the maximum number of elements present in the stack at any point during evaluation?

- A) 3 elements
- B) 4 elements
- C) 5 elements
- D) 6 elements

Correct Answer: B) 4 elements

Explanation: Operands are pushed onto the stack sequentially. The maximum stack depth occurs when evaluating the sub-expression values. Pushing 6, then 2, then 3 gives a depth of 3. After evaluation, operations continue until a sequence of operands like 3, 8, 2 is accumulated over previous results, hitting a maximum of 4 nodes simultaneously on the stack.

Section 2: Queue Data Structure (Questions 18-33)

Q18 [GATE CSE 2012]

A circular queue of capacity $n - 1$ elements is implemented using an array of size n . REAR and FRONT are array index tracking variables. Initially $\text{REAR} = \text{FRONT} = 0$. What are the conditions used to detect when the queue is FULL and when it is EMPTY?

- A) Empty: $\text{REAR} == \text{FRONT}$ Full: $(\text{REAR} + 1) \% n == \text{FRONT}$
- B) Empty: $\text{REAR} == \text{FRONT}$ Full: $\text{REAR} == n$
- C) Empty: $\text{REAR} == \text{FRONT} + 1$ Full: $\text{REAR} == \text{FRONT}$
- D) Empty: $\text{REAR} == \text{FRONT}$ Full: $(\text{REAR} - 1) \% n == \text{FRONT}$

Correct Answer: A) Empty: $\text{REAR} == \text{FRONT}$ Full: $(\text{REAR} + 1) \% n == \text{FRONT}$

Explanation: When $\text{REAR} == \text{FRONT}$ the queue has no elements. To differentiate between a completely full queue and an empty queue, one array slot is intentionally left empty. The queue is considered full when incrementing REAR points directly to FRONT.

Q19 [GATE CSE 2014]

If we restrict insertions and deletions in a double-ended queue (Deque) such that elements can only be inserted at the rear but can be removed from both the front and the rear, what structural variant is created?

- A) Output-Restricted Deque
- B) Input-Restricted Deque
- C) Normal FIFO Queue
- D) Priority Queue

Correct Answer: B) Input-Restricted Deque

Explanation: Deques allow data access at both boundaries. Restricting insertions to a single end while keeping both ends open for removals creates an Input-Restricted Deque.

Section 3: Linked List Data Structure (Questions 34-50)

Q34 [GATE CSE 2010]

The following C function takes a singly-linked list as input, moves the last element to the front of the list, and returns the updated head pointer. What code correctly fills the blank line?

```
Node *move_to_front(Node *head) {  
    Node *p, *q;  
    if ((head == NULL) || (head->next == NULL)) return head;  
    q = NULL; p = head;  
    while (p->next != NULL) { q = p; p = p->next; }
```

```
// LINE BLANK  
}
```

- A) $q \rightarrow \text{next} = \text{NULL}$; $p \rightarrow \text{next} = \text{head}$; $\text{head} = p$; return head;
- B) $p \rightarrow \text{next} = \text{head}$; $\text{head} = p$; return head;
- C) $q \rightarrow \text{next} = \text{head}$; $\text{head} = p$; return head;
- D) $p \rightarrow \text{next} = q$; $q \rightarrow \text{next} = \text{NULL}$; return head;

Correct Answer: A) $q \rightarrow \text{next} = \text{NULL}$; $p \rightarrow \text{next} = \text{head}$; $\text{head} = p$; return head;

Explanation: To move the last node p to the front, its predecessor q must become the new tail by setting $q \rightarrow \text{next} = \text{NULL}$. Then p is linked to the old head via $p \rightarrow \text{next} = \text{head}$, and the head is updated to p .

Q35 [GATE CSE 2016]

A queue is implemented using a non-circular singly linked list with a head and a tail pointer. Enqueue inserts a new node at the head, and dequeue removes a node from the tail. What is the worst-case time complexity of these operations?

- A) Enqueue: $O(1)$; Dequeue: $O(1)$
- B) Enqueue: $O(1)$; Dequeue: $O(n)$
- C) Enqueue: $O(n)$; Dequeue: $O(1)$
- D) Enqueue: $O(n)$; Dequeue: $O(n)$

Correct Answer: B) Enqueue: $O(1)$; Dequeue: $O(n)$

Explanation: Inserting at the head takes constant time ($O(1)$). However, removing from the tail requires updating the tail pointer to point to its predecessor node. Because this is a singly linked list, finding the predecessor requires traversing the entire list from the head, taking linear time ($O(n)$).

Q36 [GATE CSE 2012]

A singly linked list contains a loop. A fast pointer (moving 2 nodes per step) and a slow pointer (moving 1 node per step) start together from the head node. If the distance from the head to the start of the loop is X nodes, and the loop contains Y nodes, after how many iterations will they meet?

- A) $X + Y$ iterations
- B) The first integer multiple of Y that is greater than or equal to X
- C) $Y - X$ iterations
- D) X iterations

Correct Answer: B) The first integer multiple of Y that is greater than or equal to X

Explanation: The fast pointer travels at twice the speed of the slow pointer. For them to meet inside the loop, the difference in the total distance they traveled must be an integer multiple of the loop's length (Y), and they must have traveled far enough to enter the loop ($k \geq X$).

Detailed Analysis

This question tests understanding of **Floyd's Cycle Detection Algorithm** (also known as the "tortoise and hare" algorithm). Here's how it works:

- **Slow pointer** moves 1 step at a time
- **Fast pointer** moves 2 steps at a time
- Both start from the head simultaneously

The key insight is that once both pointers enter the loop, the fast pointer gains on the slow pointer by exactly 1 position per iteration. They will meet when the fast pointer has "lapped" the slow pointer by completing exactly one full cycle of the loop.

The meeting occurs at the first integer multiple of Y (loop length) that allows both pointers to have entered the loop, which must be at least X steps from the start.

Q36 [GATE CSE 2012]

A singly linked list contains a loop. A fast pointer (moving 2 nodes per step) and a slow pointer (moving 1 node per step) start together from the head node. If the distance from the head to the start of the loop is X nodes, and the loop contains Y nodes, after how many iterations will they meet?

- A) $X + Y$ iterations
- B) The first integer multiple of Y that is greater than or equal to X
- C) $Y - X$ iterations
- D) X iterations

Correct Answer: B) The first integer multiple of Y that is greater than or equal to X

Explanation: The fast pointer travels at twice the speed of the slow pointer. For them to meet inside the loop, the difference in the total distance they traveled must be an integer multiple of the loop's length (Y), and they must have traveled far enough to enter the loop ($k \geq X$).

Q37 [GATE CSE 2011]

You are given a pointer p to an intermediate node inside a singly linked list. You do not have access to the head pointer. What is the time complexity to delete node p?

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) Impossible without head pointer

Correct Answer: A) $O(1)$

Explanation: Since you cannot find the predecessor node directly, you can copy the data from the next node into node p, and then delete the next node instead. This achieves the same result in constant time.

Q38 [GATE CSE 2005]

What is the minimum time complexity and extra space required to reverse a singly linked list of n elements iteratively?

- A) Time: $O(n)$; Space: $O(n)$
- B) Time: $O(n)$; Space: $O(1)$
- C) Time: $O(n^2)$; Space: $O(1)$
- D) Time: $O(\log n)$; Space: $O(1)$

Correct Answer: B) Time: $O(n)$; Space: $O(1)$

Explanation: A linked list can be reversed in a single pass by updating each node's next pointer in place using three temporary pointer variables (prev, curr, and next), requiring constant extra space.

Q39 [GATE CSE 2018]

Which of the following statements is true regarding arrays and linked lists?

- A) Arrays provide $O(1)$ random access but have higher insertion costs due to element shifting; linked lists require linear traversal for lookups
- B) Linked lists provide $O(1)$ random access
- C) Arrays require sequential scan to find an element at a given index
- D) Both have identical insertion costs at any arbitrary location

Correct Answer: A) Arrays provide $O(1)$ random access but have higher insertion costs due to element shifting; linked lists require linear traversal for lookups

Explanation: Arrays store elements in contiguous memory, allowing direct index lookups. Linked lists use pointers scattered across memory, requiring a sequential scan to find elements but allowing fast insertions once the position is found.

Q40 [GATE CSE 2003]

What is the worst-case time complexity to sort a singly linked list of n elements using the Merge Sort algorithm?

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(\log n)$

Correct Answer: B) $O(n \log n)$

Explanation: Merge Sort splits the list in half recursively ($O(\log n)$ levels) and merges the sub-lists back together ($O(n)$ work per level). This holds true for linked lists and does not require random access.

Q41 [GATE CSE 1992]

What is the memory overhead associated with a Doubly Linked List compared to a Singly Linked List?

- A) No extra memory overhead
- B) One extra pointer field per node
- C) Two extra pointer fields per node
- D) Half the pointer fields

Correct Answer: B) One extra pointer field per node

Explanation: Each node in a singly linked list stores one pointer (next). A doubly linked list node must store two pointers (next and prev), doubling the memory overhead used for pointer tracking.

Q42 [GATE CSE 2006]

Suppose a singly linked list is used to implement a stack. Where should the push and pop operations occur to ensure optimal performance?

- A) At the tail node
- B) At the head node
- C) In the middle node
- D) At both ends alternatively

Correct Answer: B) At the head node

Explanation: Inserting or deleting a node at the head of a linked list takes constant time ($O(1)$) and does not require traversing the list, matching a stack's performance requirements.

Q43 [GATE CSE 1995]

What is the primary cause of a memory leak when working with dynamically allocated linked lists?

- A) Freeing nodes too early**
- B) Losing the reference pointer to a node without freeing its allocated memory**
- C) Using static variables inside nodes**
- D) Allocating too little memory**

Correct Answer: B) Losing the reference pointer to a node without freeing its allocated memory

Explanation: If you overwrite a node's pointer without explicitly calling `free()` or `delete`, that block of memory remains allocated in the heap but becomes unreachable by the program.

Q44 [GATE CSE 2013]

What is the purpose of using a dummy header node in a linked list implementation?

- A) To store the total number of items**
- B) To simplify code logic by eliminating edge cases for empty lists**
- C) To speed up random index traversal**
- D) To reduce memory allocation size**

Correct Answer: B) To simplify code logic by eliminating edge cases for empty lists

Explanation: A dummy node ensures that every valid data node always has a preceding neighbor, removing the need for special checks when inserting or deleting at the head of the list.

Q45 [GATE CSE 2001]

What is the worst-case time complexity to concatenate two singly linked lists of lengths m and n if you only have pointers to their head nodes?

- A) $O(1)$**
- B) $O(m)$**
- C) $O(n)$**
- D) $O(m + n)$**

Correct Answer: B) $O(m)$

Explanation: To link the two lists, you must traverse the first list from its head to find its terminal tail node, which takes time proportional to the first list's length (m).

Q46 [GATE CSE 2008]

Can a circular linked list be used to implement a queue efficiently without maintaining a head pointer?

- A) Yes, by maintaining a pointer to the tail node**
- B) No, a head pointer is strictly required**
- C) Only if list length is less than 100**
- D) Yes, but only for stacks**

Correct Answer: A) Yes, by maintaining a pointer to the tail node

Explanation: In a circular linked list, the tail node points directly to the head node ($\text{tail} \rightarrow \text{next} == \text{head}$). This gives you instant access to both ends of the queue using a single pointer reference.

Q47 [GATE CSE 2017]

Which data structure combination is ideal for implementing an LRU (Least Recently Used) cache?

- A) A Hash Map and a Doubly Linked List**
- B) A Heap and a Queue**
- C) An Array and a Binary Search Tree**
- D) A Stack and a Trie**

Correct Answer: A) A Hash Map and a Doubly Linked List

Explanation: The doubly linked list allows cache pages to be moved or removed in constant time ($O(1)$) to track usage recency, while the hash map provides fast lookups ($O(1)$) to check if a page is already in the cache.

Q48 [GATE CSE 1996]

What is a major disadvantage of linked lists compared to arrays regarding hardware cache optimization?

- A) High dynamic memory consumption**
- B) Poor CPU cache locality**

- C) Lack of pointer flexibility
- D) Complexity of deletion operations

Correct Answer: B) Poor CPU cache locality

Explanation: Arrays store elements in contiguous blocks of memory, which optimizes hardware pre-fetching. Linked list nodes are scattered across the heap, leading to frequent CPU cache misses during traversal.

Q49 [GATE CSE 2019]

What is the time complexity to find the middle node of a linked list using the two-pointer method?

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

Correct Answer: B) $O(n)$

Explanation: While the fast pointer allows you to locate the middle node in a single pass, you still need to traverse through all n nodes in the list, resulting in a linear time complexity.

Q50 [GATE CSE 2022]

How does a Skip List achieve logarithmic search times?

- A) By sorting nodes dynamically using quicksort
- B) By building an express-lane hierarchy of pointers over a standard linked list
- C) By transforming into a balanced binary tree
- D) By hashing key indexes into memory blocks

Correct Answer: B) By building an express-lane hierarchy of pointers over a standard linked list

Explanation: Skip lists use multiple layers of forward pointers that skip over blocks of nodes. This allows search queries to bypass large sections of data, mimicking a binary search over a linked list and reducing search times to $O(\log n)$.